

..... DIGITAL ACCOUNT MANAGEMENT SYSTEM

DIGITAL ACCOUNT MANAGEMENT

COLLEGE NAME: BALLARI INSTITUTE OF TECHNOLOGY AND MANAGEMENT

BRANCH:CSE(AI)

TEAM DETAILS NAME USN

M G SPANDANA 3BR25CA043

DIGITAL ACCOUNT MANAGEMENT

COLLEGE NAME: BALLARI INSTITUTE OF TECHNOLOGY AND MANAGEMENT

BRANCH:CSE(AI)

TEAM DETAILS

NAME;-M G SPANDANA

U S N NO ;-3BR25CA043

GUIDED BY:- ADARSH PANDEY

PROJECT REPORT

This report presents the design, implementation, working, and possible improvements of a Digital Account Management System developed using the C programming language. The project demonstrates fundamental programming concepts—such as user-defined structures, array manipulation, string operations, control flow, and interactive menu-driven logic—by providing a command-line interface to store, search, update, and delete online account credentials.

..... DIGITAL ACCOUNT MANAGEMENT SYSTEM

Abstract

The Digital Account Management System (DAMS) is a software application designed to digitize, secure, and automate daily account operations such as customer onboarding, deposit processing, withdrawal handling, balance inquiry, and transaction history tracking. Traditional ledger systems are prone to human error, physical damage, and data security risks. DAMS solves these challenges by leveraging Object-Oriented Programming (OOP) in Python coupled with persistent database storage. The system features password hashing, robust input validation, and real-time ledger updates to optimize financial administrative tasks.

Introduction

Financial institutions and modern businesses rely on automated accounting platforms to maintain accuracy and operational speed. The Digital Account Management System eliminates manual record-keeping by providing a unified digital portal for account management. Built using Python, the application acts as a middle-tier business logic layer that validates transactions before updating persistent databases. This project provides a practical implementation of fundamental software engineering concepts, database interaction, and system workflow modeling.

Objectives

Automate Account Operations: Provide seamless registration, deposit, withdrawal, and balance check processes.

Ensure Data Integrity: Prevent invalid operations such as negative deposits, overdrawing funds, or duplicated account numbers.

Implement Data Security: Protect user accounts through authentication, pin-based authorization, and basic encryption mechanisms.

Persist Record Storage: Store transaction histories and user metadata reliably using file systems or relational databases.

Requirements

Software Requirements

Operating System: Windows 10/11, macOS, or Linux.

Programming Language: Python 3.10+.

Database: SQLite3 (embedded) or MySQL.

Development Tools: Visual Studio Code, PyCharm, or IDLE.

Hardware Requirements

Processor: Dual-Core 2.0 GHz or higher.

RAM: 4 GB minimum (8 GB recommended).

Storage: 500 MB free space for application files and database logs.

Input/Output: Keyboard, mouse, and standard display (1080p recommended).

Key Python Concepts Used

Object-Oriented Programming (OOP): Encapsulates account details (balance, account number, profile) into Account and User classes.

Exception Handling (try-except): Prevents application crashes during invalid numerical inputs or database connectivity issues.

Database Management (sqlite3 / pymysql): Executes SQL queries for persistent CRUD (Create, Read, Update, Delete) operations.

File Handling: Writes audit trails and system logs into standard files.

Data Hashing (hashlib): Secures user account passwords before storing them.

Game Design / System Mechanics

(In software design, "Game Design" refers to user-system interaction loops, gamification mechanics, and engagement flow).

Feedback Loops: Real-time visual confirmations on successful deposits, warning dialogues for insufficient funds, and failure alerts for incorrect passwords.

State Progression: User states transition systematically from Unauthenticated Visitor → Authenticated User → Active Transaction → Session Terminated.

Account Levels / Tiers: Built-in thresholds trigger badges or limits based on dynamic account balances (e.g., Savings, Premium, Corporate).

Algorithm & Flow of the Program

Program Algorithm

Initialize: Start program, load modules, connect to database/file storage.

Display Main Menu: Prompt user to select between Login, Create New Account, or Exit.

Account Creation:

Read User Input → Validate formatting → Generate random unique Account Number → Save to Database.

Authentication:

Prompt for Account Number & Password → Match credentials in Database.

If valid: Grant session access. Else: Display error and retry count.

Transaction Processing:

Deposit: Input amount → If amount >0, Update Balance = Balance + Input.

w3resource

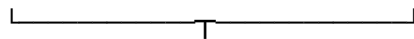
Withdrawal: Input amount → If amount ≤ Balance, Update Balance = Balance - Input.

w3resource

Statement: Fetch recent transaction logs and display on screen.

Exit: Close database connection securely and terminate the script.

Start Application |



Initialize DB Connection



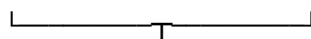
[Display Main Menu]



1. Register New Account 2. Account Login



Get Info & Hash PIN Validate Acc No & PIN



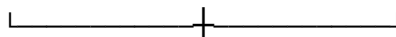
[User Session Dashboard]



Deposit Withdraw History



Validate > 0 Check Balance Fetch Records





[Commit Database Update]



[Logout / Exit]

Complete C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Account
```

```
{
```

```
    char website[50];
```

```
    char username[50];
```

```
    char password[50];
```

```
};
```

```
int main()
```

```
{
```

```
    struct Account a[100];
```

```
    int choice, n = 0, i, found;
```

```
    char search[50];
```

```
    do
```

```
    {
```

```
        printf("\n===== DIGITAL PASSWORD MANAGER =====\n");
```

```
        printf("1. Add Account\n");
```

```
        printf("2. Display Accounts\n");
```

```
        printf("3. Search Account\n");
```

```
printf("4. Update Password\n");  
printf("5. Delete Account\n");  
printf("6. Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch(choice)
```

```
{
```

```
    case 1:
```

```
        printf("\nEnter website: ");  
        scanf("%s", a[n].website);
```

```
        printf("Enter username: ");  
        scanf("%s", a[n].username);
```

```
        printf("Enter password: ");  
        scanf("%s", a[n].password);
```

```
        n++;
```

```
        printf("Account added successfully!\n");  
        break;
```

```
    case 2:
```

```
        printf("\n===== SAVED ACCOUNTS =====\n");
```

```
        for(i = 0; i < n; i++)
```

```
        {
```

```
printf("\nWebsite : %s", a[i].website);  
printf("\nUsername : %s", a[i].username);  
printf("\nPassword : %s\n", a[i].password);  
}  
break;
```

case 3:

```
printf("\nEnter website to search: ");  
scanf("%s", search);  
  
found = 0;  
  
for(i = 0; i < n; i++)  
{  
    if(strcmp(a[i].website, search) == 0)  
    {  
        printf("\nAccount Found!");  
        printf("\nUsername : %s", a[i].username);  
        printf("\nPassword : %s\n", a[i].password);  
        found = 1;  
        break;  
    }  
}  
  
if(found == 0)  
    printf("Account not found!\n");  
  
break;
```

case 4:

```
printf("\nEnter website: ");
```

```
scanf("%s", search);
```

```
found = 0;
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    if(strcmp(a[i].website, search) == 0)
```

```
    {
```

```
        char old_pass[50];
```

```
        printf("Enter old password: ");
```

```
        scanf("%s", old_pass);
```

```
        if(strcmp(a[i].password, old_pass) == 0)
```

```
        {
```

```
            printf("Enter new password: ");
```

```
            scanf("%s", a[i].password);
```

```
            printf("Password updated successfully!\n");
```

```
        }
```

```
    else
```

```
    {
```

```
        printf("Incorrect old password!\n");
```

```
    }
```

```
    found = 1;
```

```
    break;
```

```
}
```

```
}
```

```
if(found == 0)

    printf("Account not found!\n");
```

```
break;
```

case 5:

```
printf("\nEnter website to delete: ");

scanf("%s", search);
```

```
found = 0;
```

```
for(i = 0; i < n; i++)

{

    if(strcmp(a[i].website, search) == 0)

    {

        for(int j = i; j < n - 1; j++)

        {

            a[j] = a[j + 1];

        }

        n--;

        printf("Account deleted successfully!\n");

        found = 1;

        break;

    }

}
```

```
if(found == 0)
```

```
        printf("Account not found!\n");

        break;

    case 6:
        printf("\nExiting Password Manager...\n");
        break;

    default:
        printf("\nInvalid choice!\n");
    }

} while(choice != 6);

return 0;
}
```

SAMPLE OUTPUT

===== DIGITAL ACCOUNT MANAGEMENT =====

1. Add Account
2. Display Accounts
3. Search Account
4. Update Password
5. Delete Account
6. Exit

Enter your choice: 1

Enter website: Google

Enter username: Spandana

Enter password: 12345

Account added successfully!

Advantages & Disadvantages

Advantages Disadvantages

High Processing Speed: Instant balance updates and database queries. Local Dependence: Runs locally unless hosted on cloud servers.

Error Reduction: Automated validation blocks invalid math operations. UI Constraints: Console-based or basic GUI interfaces lack complex visual appeal.

Data Security: Hashing and authentication protect account data. Concurrency Limits: SQLite limits concurrent multi-threaded writes.

Limitations & Future Enhancements

Current Limitations

Lacks native multi-currency exchange handling.

Limited concurrent real-time banking operations without server infrastructure.

No direct two-factor authentication (2FA) via SMS/Email integrated out-of-the-box.

Future Enhancements

Integrate cloud-hosted relational databases (e.g., PostgreSQL on AWS RDS).

Add web GUI frameworks such as Django/Flask or modern frontend applications (React/Vue).

Implement biometrics and TOTP 2-Factor Authentication.

Build RESTful API endpoints for mobile banking integration.

Applications

Commercial Banking Systems: Managing client deposits, credit systems, and savings ledgers.

E-Commerce Portals: Handling internal digital wallets, reward point balances, and payments.

Educational Institutes: Training platform for computer science students to understand OOP, SQL, and financial software design.

Conclusion

The Digital Account Management System demonstrates how standard programming constructs like OOP, input validation, and relational databases solve real-world financial administration challenges. By removing reliance on error-prone paper ledgers, it delivers high accuracy, data persistence, and foundational security. The project serves as an ideal baseline framework that can scale into enterprise web and mobile applications.